Final Project: Pushdown automata and sentence-processing

Willie (Chun Wei) Wen, *University of California, Los Angeles*

Abstract

Syntax, of how words form phrases and sentences, in Formal language theory has been a main vital role in computational linguistics. Context-Free Grammars(CFGs) give a neutral role for human language syntax to be described. This Project involves the implementation of various parsing algorithm methods in pushdown automata (PDAs) in Haskell about CFG-to-PDA conversions. This computational application of exploring model suitability pros and cons with linguistics knowledge and computer science would provide a possibility to evaluate words in natural language efficiently and systematically.

1. Introduction

Computational linguistics is an interdisciplinary field of approaching linguistics from a computational aspect to solve questions with linguistics data computationally. Finite state automata (FSA) is a computational model which can be an imitation of syntax in computers to generate natural language. However, practical natural language processing tends to be more complicated, which occurs with recursive structure with ambiguity that exceeds FSAs processing ability. Therefore, we must use pushdown automata (PDA) - a stronger model which has Stack depth to address the logics of language with proper order. This project would discuss different methods of parsing: bottom-up, top-down, and left-corner in Haskell. The statically typed functional programming language Haskell is ideal for linguistics grammar models because of its well symbolic and recursive processing. A program would be built for basic analysis of syntax rules, with utilization of CFG-PDA through algorithms with syntax rules.

1.1. Pushdown Automata

A use of **pushdown automaton (PDA)** is preferred over FSA in this project because PDA incorporates a Stack which is a form of memory load. A PDA consists of the stack alphabets, the surface alphabets, a finite set of initial stack contents, a finite set of ending stack contents, and a finite set of transitions. A PDA is able to process recursive grammar structures by Stack memory which is beyond the finite control. A PDA works by processing an input symbol(or sometimes an ϵ-transition that doesn't consume input) than modify the Stack, and move to the next state by transitions rules. The Stack makes PDA able to add/push, delete/pop or unchange depending on transition rules in it. This project we would use multiple parsing methods to implement PDAs from their CFGs form in Haskell, specifically modify the PDA consisting of Initial stack, Final stack, and finite set of transitions.

1.2. Syntax rule

A syntax rule is a formal theory expressing syntactic structures in natural language. It is the foundation of how words or morphemes could generate grammatical phrases or sentences. In this project, syntax rule is illustrated by CFG: type CFG nt sy = ([nt], [RewriteRule nt sy])

data RewriteRule nt sy = TRule nt sy | NTRule nt (nt,[nt]) deriving (Show,Eq)

TRule is a terminal rule that makes a nonterminal rewritten as a terminal(word). NTRule is a nonterminal rule that makes a nonterminal rewritten as another nonterminal followed by an optional set of nonterminals. Each rule defines how a nonterminal symbol can be once represented to a sequence of words (terminal) with possible other nonterminals. The syntax rules and words we constructed here are only for this project. They are both derived from English, though they're not the case practically, as actual generative grammar with movements and constraints would require more advanced linguistic and computer science theory, models and tools.

2. Function Design

Following functions are built to evaluate and implement

CFG-PDA parsing strategies.

2.1. completeSequences

This function is the exact core factor of the implementation of PDA, responsible for finding all the transition sequences systematically with Stack. For Base Case, which when the input string is empty, has two parts. First is to check if currentStack is in finalStacks, if it is then seq is a valid final transition sequence and added to results. Otherwise there's no valid transition in the path. Second is about ϵ-transitions, which is the transition that does not consume input. It goes through all the PDA transitions(delta), and selects the one where input symbol is Nothing then applies tryTransition to update its Stack; if the result is valid under fn, the function recursively extends the sequence with this transition. For Recursive Case, which when the input string is not empty, has two parts as well. Before introducing both parts we have to extract the first symbol a from the input (a:as) as a is the current symbol, and as is the remainder. First is to check the ϵ-transitions with (a:as), each transition tr where the input symbol is Nothing, we check if it can apply (tryTransition) and recursively call completeSequences, extending the sequence without consuming a.Second is to check normal input transitions, those transitions,those transitions must match the next input symbol a. Iterate over transitions tr where the input symbol is Just b. If b == a, the transition matches the expected input. If tryTransition valid, the new stack ns is

checked against fn. If valid, we recursively call completeSequences, consuming a and proceeding with as. Some crucial points here are that ϵ-transitions are always processed first to make sure all the possible paths are discovered, this is especially crucial in ambiguous grammars where there are multiple paths to parse. fn let no invalid stack are reached, crucial for restricting parsing depth.

2.2. tryTransition

This function tries to apply transition tr to current stack s. If it doesn't apply(popped), function returns Nothing means the Stack symbol doesn't match the top of s.If it applies (pushed), it removes (popped) from s and pushes a new symbol to Stack and returns Just s'.

2.3. checkSequence

The function recursively applies each transition using tryTransition to process the sequence to confirm a transition sequence is internally consistent.

2.4. pdarecognize

pdaRecognize pda fn w finds all the transition sequences by which pda generates/accepts the string w, with fn specifying an out-of-bounds restriction in the same way as above for completeSequences.

2.5. Bottom-Up Parsing

Converts CFG into a bottom-up PDA. The PDA builds structures from the bottom to top, which means from an empty Stack to the final S nonterminal symbol of the syntax. Bottom-up has two kinds of transitions. First is the shift transition which input symbol into Stack, when a terminal symbol appears, pushing its corresponding nonterminal into stack. Second is the reduce transition which applies syntax rules, if the nearest component of Stack (Or also called-top of the stack, same concept) obeys the right-hand side rule (explanation below), convert and integrate them with the corresponding nonterminal. A specific takeaway point to understand this parsing method is that it follows the given linguistic syntax rules first before reducing them into their corresponding nonterminal, which could be interpreted as the so-called right-hand side rule.

2.6. Top-Down Parsing

Converts CFG into a top-down PDA. The PDA builds structures from the bottom to top, which means from nonterminal syntax symbol S on Stack to an empty Stack. Top-Down has two kinds of transitions. First is the match transition, when a terminal symbol is at the nearest stack and it matches the current input symbol, pop (delete) it from the stack. Second is the predict transition, when a nonterminal symbol is at the nearest stack, convert and disintegrate it to terminal symbols with the foundation of syntax rules and right-hand side rule.

2.7. Left-Corner Parsing

Converts CFG into a left-corner PDA. This parsing

method has combined the two parsing methods(bottom-up/top-down) above, however, left-corner parsing would recognize the **leftmost** symbol before fully finishing predicting the rest in the Stack. Furthermore, there's this most important technical approach to the structure of Stack that other two parsing methods don't have, the significance of Stack representation: Predict nt and Found nt. Predict nt represents a nonterminal that is predicted but has not satisfied and finds every symbol to reach its final form. Found nt represents a nonterminal that is confirmed it has found every it belongs to and reaches its final form, and it can also represent a terminal symbol for input. Initial Stack starts with Predicted S since we predict S which is the highest level in syntax rule yet we find every symbol to satisfy, while Final Stack ends with empty. There are four transition rules, that shift and match transitions are similar above. One of the rest is LcPredict transition, that if a rule has a leftmost nonterminal, expand only the leftmost symbol first, that after a Found push more Predict. The other is LcConnect transition, which connects a found leftmost symbol with an existing prediction. If the stack has Found A followed by Predicted B, and there's a rule B → A y, we pop both and push y as Predicted. This continues the expansion of B after confirming that A is its leftmost symbol. Predict and Found must be correctly structured in the transitions in order to achieve all the possible paths. The reason why the other two functions above didn't use this is because bottom-up operates by terminal reductions hence no predictions, top-down distinguishes nonterminal to expand and terminal to match naturally.

2.8. Bottom-Up/Top-Down/Left-Corner Recognize

All the recognize functions behave similarly except the Stack constraint part. While Bottom-up is naturally bounded by its structure because it does not predict, hence there's never an overprediction problem so it does not grow unpredictably. Both Top-Down and Left-Corner involve prediction. Expanding nonterminals before consuming input could lead to overprediction or Infinite recursion to the completeSequences function. Furthermore, Left-Corner's Stack still grows in ambiguous/recursive grammars so we need to limit its depth of partial predictions. We couldn't inhibit the limits of prediction internally, therefore we must implement a constraint to prune the inevitably excessive part to keep the size of our whole output and Stack finite. To achieve this, we implement (length stack <= length input + 1) into the corresponding recognize functions. This could also be interpreted as an imitation of humans that we also don't check for all possible syntactic continuations for a sentence, we prune our mindset based on context, memory and plausibility.

All the parsing methods successfully identify all of the

possible analyses of a string, which include multiple trees that have to do with structural ambiguity and potential different derivation orders by syntax rules.

3. Analysis

3.1. Stack Depth

/.Left-Embedding Stack

E.g.”the boss’s actor’s baby won ” (Nested NP)

Top-Down & Left-Corner: Each NP prediction delays consumption, Stack grows linearly along with the number of left-embedded NPs.

Bottom-Up: Stable, shift terminals then reduce afterwards.

/.Right-Embedding Stack

E.g.”John met the actor that won the award”

(VP → V NP, SRC → THAT VP)

Top-Down: smaller Stack after NP prediction, SRC comes after it and processes.

Left-Corner: Process "the actor" first then SRC → VP.

Bottom-Up: Form structure from "award" and "won" back to higher nodes.

/.Center-Embedding Stack

E.g.”the boy the boss met won”

Top-Down: Predicts NP → D N ORC → ORC → NP V → NP → D N before consuming input. Stack grows fast notoriously: each embedded NP prediction adds more items.

Left-Corner: Stack still grows by delayed predictions.

Bottom-Up: Shifts "the boy the boss met" before reduction — Stack depth is still high.

.Reflection on Finite-State Automata (FSA) Insufficiency

FSA has no Stack as memory, therefore it cannot work on recursive grammars, nesting phrases and more such as:

Left-Embedding: E.g.”the boss’s actor’s baby won ” - recursive nesting of NP required → NP POSS N.

Right-Embedding: E.g.”John met the actor that won the award” - Needs to track sub clauses like SRC → THAT VP

Center-Embedding: E.g.”the boy the boss met won”

→ Dynamic track of various to-be-satisfy constituents required — impossible without a stack.

3.2. CFG as a model of human grammars

Context-Free Grammar is indeed formal, but still considered an approximation to actual human grammars. It captures constituency and syntactic structures systematically very well. There are two issues. First is it could cause overgeneration of semantically unacceptable sentences, yet it may be syntactically acceptable. Second is about center-embedding, which is difficult for human brain processing rather than a grammar issue. Therefore to utilize CFG for human language it must have bounded Stack depths, hierarchical semantics interpretation and

even preference for left or right branching structures. To study and understand a particular field, we must study the field as a whole, at least in terms of language, syntax is highly related to semantics and a lot more which cannot be neglected to have deeper comprehension.

3.3. Difficulties

/.“out of bounds” filter to output

The core search engine of the whole project would be one of the significant challenging parts to execute. Specific part of it would be not the input-consuming transitions and the more conceptually abstract ϵ-transitions, but the “out of bounds” filter. At first it was not obvious how to inhibit the excessive growth of output of top-down or left-corner parsing. I eventually wrote the solution (length stack <= length input + 1) into the corresponding recognize functions as its memory bound to prevent irrelevant or too “deep” transitions while maintaining correctness. I realize this might not be the perfect way to tackle this issue, however I am aware that making real progress to prune the problematic issues is good enough for basic problem solving. Every single improper matching to stack updates based on transitions logic would cause bugs which certain valid paths cannot be found.

/.leftCornerPDA

I encountered difficulties during accomplishing this function because I didn’t fully understand how the custom symbol type LCS nt, Predict and Found should be placed in the transition rules, especially the LcPredict line due to skimming the guidelines. I initially put both (map Predicted rest) ++ [Predicted lhs] instead of the correct (map Predicted rest) ++ [Found lhs] in it. It creates a bug for my function that it cannot output all the valid parses and it got me stuck for a few days. Thanks to the TA, Kalen in this class who gave me an enlightenment for me to figure things out, even it just a reminder to me for fundamental concepts, without him there’s a chance that I’d take so much more time locating the issue because my attention started to distract to the functions above and wasn’t at the place where I’m supposed to find out at all at the time. Then finally combining them with the left-corner parsing idea turned out not so surprising which took me less time when things are crystal clear.

/.CFG-to-PDA translation & Ambiguity

Converting CFG rule into PDA transitions turned out pretty smooth by the following course material throughout the quarter. However, handling the ambiguous grammar still makes it worth putting into the difficulties category. For every possibility to be found in the system, completeSequences must be recursively exploring every path include input-consuming transitions and ϵ-transitions, afterwards prune the output when the out of bounds filter is called from the recognize function (if it’s

applicable). Therefore strings like "aaa" in pda_ambig have multiple TransitionSeq which gives this project a truly elevated significance for us to have a glimpse of human language phenomena such as syntactic, attachment, or coordination ambiguity in computational processing.

4. Loose Ends/Next Steps

4.1. ϵ-transitions in CFG Assumption

PDA allows ϵ-transitions, if we put A → ϵ to CFG, it adds complexity to the whole conversion because it handles another rule that without Stack changing or input consuming. It would lead to higher chance of infinite recursion (dead loop) by generating more ambiguous paths even if the out of bound filter is enforced.

4.2. Build Trees from final TransitionSeq Reversely

If we would like to construct syntactic structure ourselves from the transitions in this project, we must discuss the pros and cons about different parsing methods. Each TransitionSeq would give us the information about its Stack's before and after, we then scrutinize backwardly from the end of the sequence and construct the tree by how a transition matched a terminal which means a leaf node in the tree, or combined symbols to a constituent. When encountering NTRule, match the terminals (D, N) to subtree (NP). When encountering TRule, create a subnode. **Top-Down** parsing would be the most intuitive which makes it easiest to do so. Because of its predictions order mimics the tree structure that gives either a terminal(leaf) or a nonterminal(internal node/subtree) as a result. **Bottom-Up** would be the hardest to do so. Because we are kind of not able to see the whole picture simply by one transition, there's only one reduction when encountering the full rhs, hence it requires tracking back to find the corresponding subnodes for each reduction which makes it difficult to retrieve. Left-Corner parsing is in between since it combines the two parsing strategies above, it recognizes the leftmost symbol then predicts and completes the rest, this gives us some information yet it requires matching Predict and Found nonterminal to their grammar rules.

4.3. Multiple Paths' Order in pdaRecognize

The path order results are based on depth-first search. Transitions listed earlier in delta are discovered first, so the earliest returned sequences tend to demonstrate local,greedy decisions such as early prediction or shifting. This can also be interpreted that it mimics human sentence processing preferences in terms of psycholinguistics. Easier,more immediately intuitive (incrementally) structures which require less memory load will be subconsciously selected. First parse of **top-down** parsing is the most straightforward, general predictions, predict transitions are first, the way it builds structure early gives us the evidence of why the

garden-path effects occur. First few parses of **bottom-up** tell the greedy local structure constructions, it prefers shift and comes before reduce, to mimic human read a little more words then combines the information. First few parses that appear early in the **left-corner** would be the one which identifies early constituents as soon as possible, match comes before predict which gives us more balanced efficiency like human incremental interpretation.

Conclusion & Reflection

This project strengthens my foundation of FSAs, CFGs, PDAs, and parsing/embedding strategies. I learned the pros and cons plus each of their purposes and the timing to utilize them, finally essentially examined computational linguistics in a macro way as a whole. Where FSAs are limited to recursive structures, but definitely necessary to exist and could be applied to lots of cases with advantages.

The presence of PDAs vitalizes CFGs which represent recursive syntax, by adding stack tracking dependencies that have yet bounded. The given outputs of Stack prove the theoretical differences by the recursive natural language processing in the model.

Generally speaking, we've built functions to generate pushdown automata from context-free grammar for particular syntax rules, and traverse the possibility of applying various parsing strategies to analyze a sentence syntactically, which is the application of syntax in computational linguistics. For analysis of syntax rules, both pushdown automata and context-free grammar are useful for us to study its computational complexity and derivational behavior. How different parsing strategies could be interpreted in numerous aspects is intriguing, and valuable to me that makes me appreciate and understand computational linguistics more as a whole.

Reference

[1] Tim Hunter. *Week7 Handout: Pushdown automata (and context-free grammars)*. [Online]Available from: https://bruinlearn.ucla.edu/courses/200764/files/19786301?module_item_id=7221357

[2] Tim Hunter. *Week6 Handout: Context-free grammars*. [Online] Available from: https://bruinlearn.ucla.edu/courses/200764/files?preview=1978